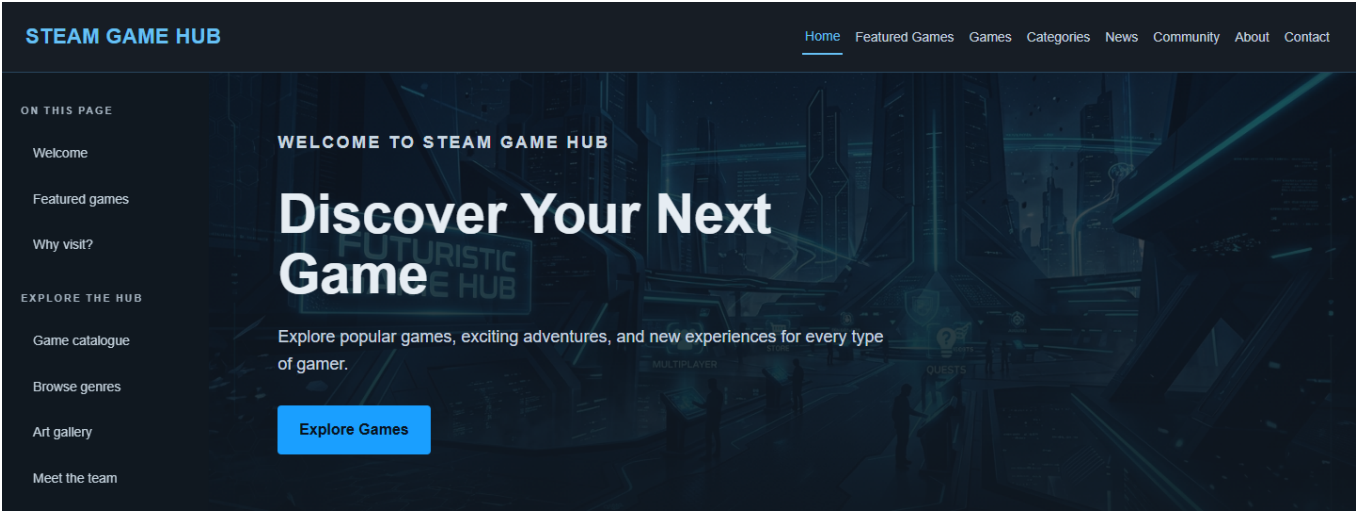


Steam Game Hub

Flexbox & Grid

A responsive, eight-page student website with shared navigation, flexible cards, named Grid areas and an original image gallery.



Team Eggs | Group SE-2507

Team member	Team member
Ташкенбаев Нурсултан	Бигали Абулхаир
Султанкулов Шынгыс	Лопаткин Артем

Objective. Apply modern CSS layout techniques to the existing Assignment 1 website, using Flexbox and Grid without float-based layout or external CSS frameworks.

Published website:

<https://abilkhayrbigali-design.github.io/Web-assignment2/>

Source repository:

<https://github.com/abilkhayrbigali-design/Web-assignment2>

Report contents: Tasks 1-4, integration and responsive checks, publishing, and final reflection. Screenshots show the completed project.

Flexbox navigation and cards

Task 1 - Navigation Bar. The existing navigation was retained and standardized across all eight pages. The logo and list of links are direct children of a Flexbox container. Space-between places them at opposite edges; center aligns them vertically. The link list also uses Flexbox with wrapping and gaps.

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 24px;
}
```



Figure 1. Desktop navigation: logo on the left, links on the right.

Task 2 - Card Row. The card containers on Home, Featured Games and Games were changed to Flexbox. Each card includes an image, title, description and a real navigation link styled as a button. The container wraps with a 24px gap. Stretch provides equal height within each row; a column Flexbox inside each card and an automatic top margin keep buttons at the bottom.

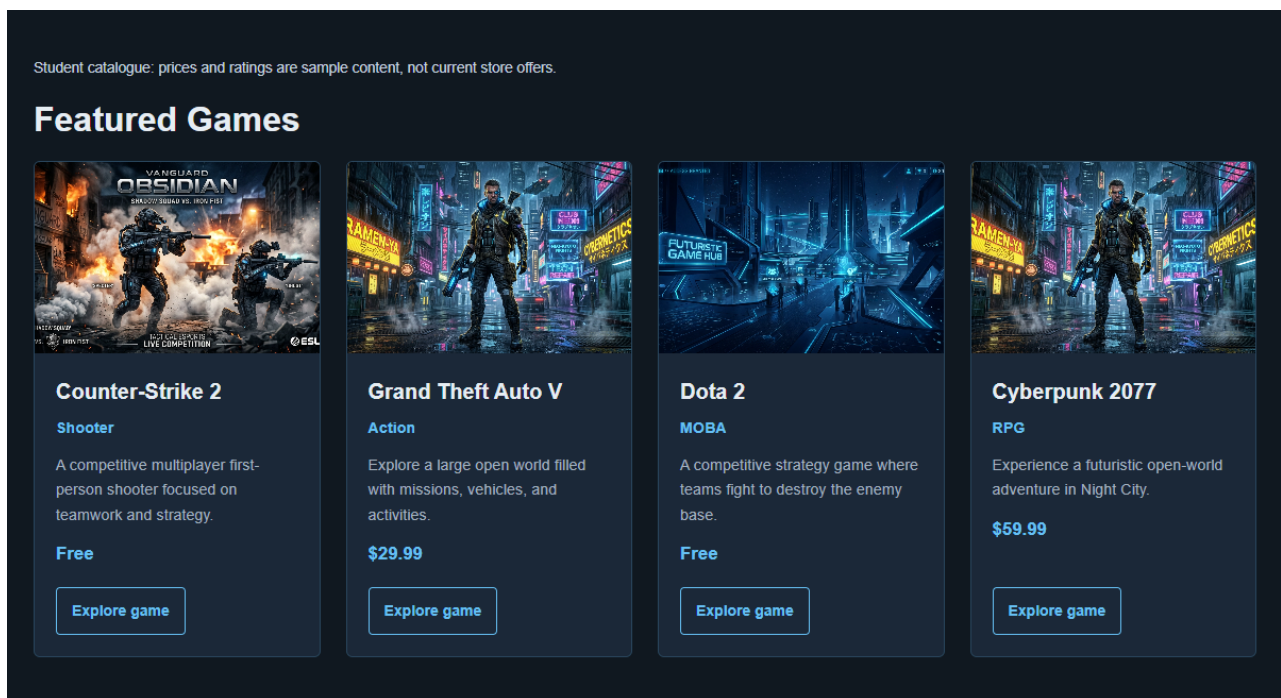


Figure 2. Four Home cards with consistent gaps and aligned action links.

Steps. 1. Use the shared card-row class. 2. Set a four-card width with $\text{calc}((100\% - 72\text{px}) / 4)$. 3. Stretch the inner content with flex: 1. 4. Add working links. 5. Apply a small upward transform, shadow and border highlight on hover. 6. Use media queries for two and one column.

Page layout with Grid areas

All eight pages share one parent Grid with four named areas. The sidebar is 220px wide; `minmax(0, 1fr)` lets the main content use the remaining width without forcing horizontal overflow. Header and footer span both columns.

```
grid-template-columns: 220px minmax(0, 1fr);
grid-template-rows: auto 1fr auto;
grid-template-areas:
  "header header"
  "sidebar main"
  "footer footer";
```

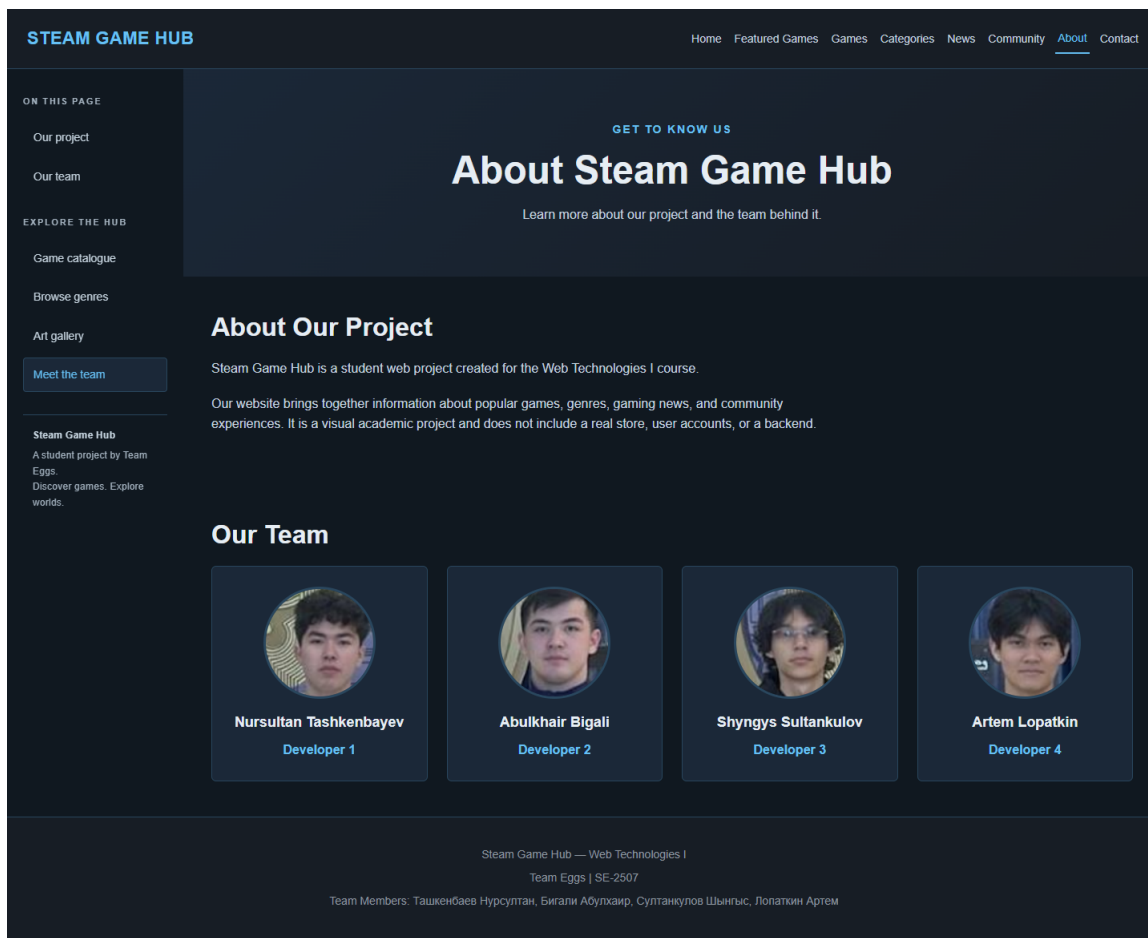


Figure 3. About page: full-width header and footer, left sidebar and right main area.

Steps. 1. Make the body a Grid container. 2. Define columns, rows and named areas. 3. Assign grid-area to the header, sidebar, main and footer. 4. Add useful section links to the sidebar. 5. At 1200px and below, switch to one column in the order header, sidebar, main, footer.

The Grid has a minimum height of 100vh. Its middle row can expand so the footer also remains at the bottom on shorter pages. Sidebar links and page navigation continue to use Flexbox inside the Grid structure.

Nine-image community gallery

The Community page contains nine separate local SVG illustrations: Neon City, Orbital Station, Forest Trail, Desert Ruins, Frozen Frontier, Sky Islands, Cyber Arena, Ocean Depths and Moonlit Keep. These original game-inspired illustrations are loaded through image elements with descriptive alternative text.

```
.gallery-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  grid-auto-rows: 1fr;
  gap: 20px;
}
```

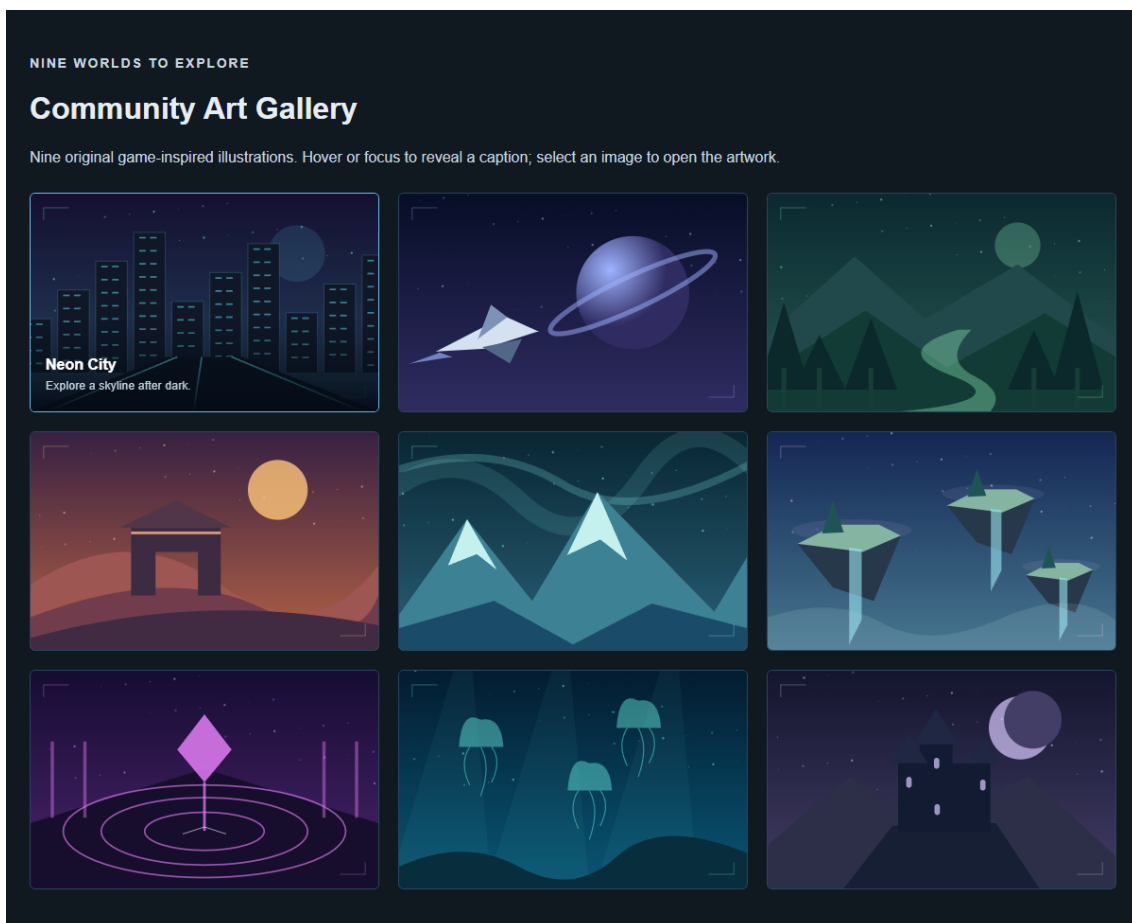


Figure 4. A three-column gallery; the first image shows its hover caption.

Steps. 1. Create nine distinct image files. 2. Place image and figcaption inside each gallery link. 3. Define equal columns and rows, with 20px gaps. 4. Keep an 8:5 image ratio. 5. Reveal the overlay with opacity and transform on hover or keyboard focus. 6. Display captions permanently on touch devices and small screens.

The gallery changes to two columns at 1000px and one at 620px. Each artwork link opens its corresponding SVG file.

Responsive behavior and verification

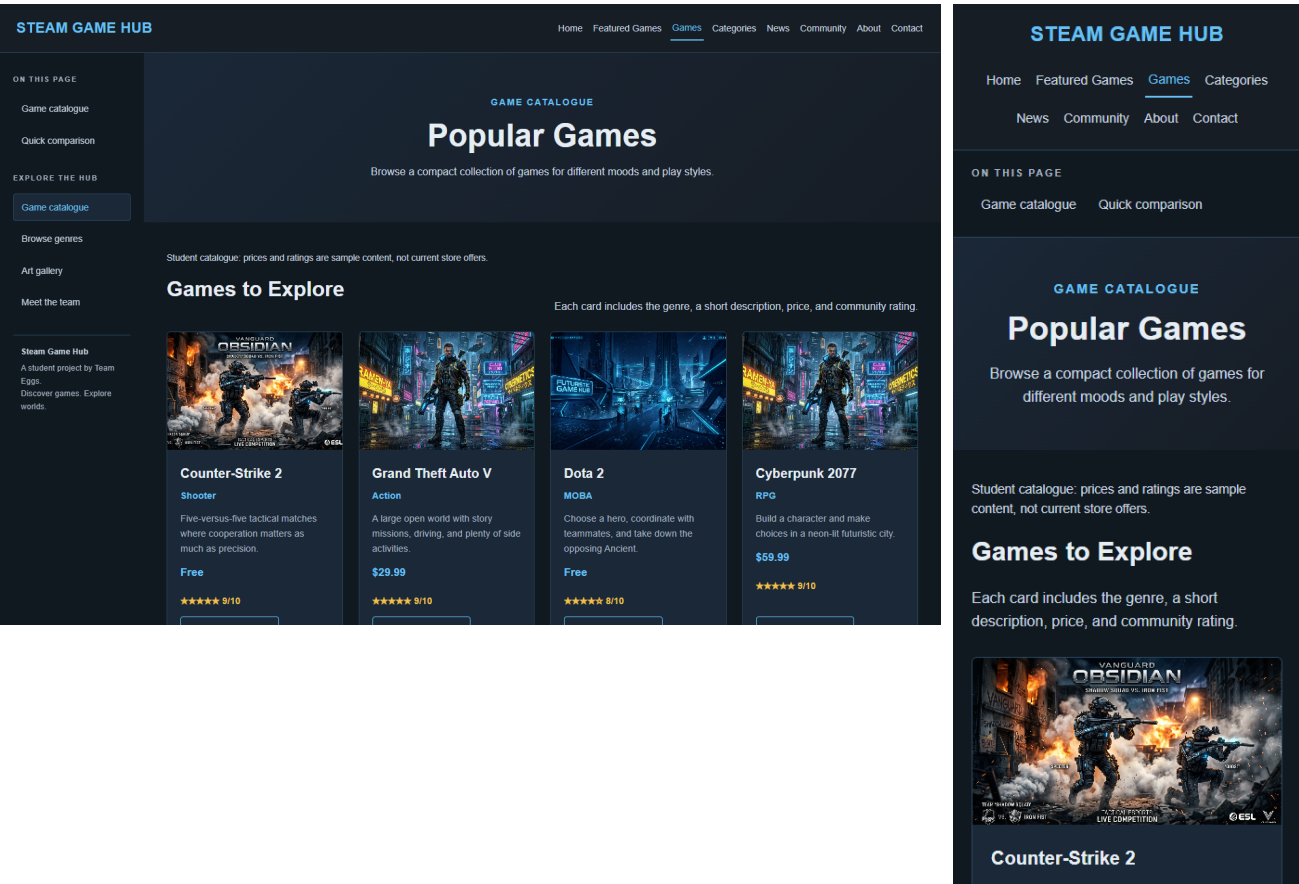


Figure 5. The Games page on desktop and at a 375px viewport width.

Component	Wide screen	Smaller screens
Outer page Grid	Sidebar + main	One column at 1200px
Game cards	4 per row	2 at 1000px; 1 at 620px
Gallery	3 per row	2 at 1000px; 1 at 620px
Navigation	Logo and links in a row	Stacked and wrapped at 1000px

Verification. All eight pages passed checks at 1440, 1280, 1201, 1200, 1024, 1000, 768, 621, 620, 375 and 320px - 88 page/viewport combinations. Checks covered overflow, images, links and fragment targets, Grid placement, Flexbox card/button alignment and equal gallery cells.

Interaction checks passed for game links, gallery hover and keyboard captions, artwork links and both demonstration forms. Native validation blocks empty required fields; valid input produces local feedback without transmission. No browser script errors were observed.

Repository and deployed website

Website:

<https://abilkhayrbigali-design.github.io/Web-assignment2/>

Repository:

<https://github.com/abilkhayrbigali-design/Web-assignment2>

Steps. 1. Prepare the static HTML, CSS, JavaScript and local image files. 2. Commit them to the shared repository. 3. Push the main branch. 4. Configure GitHub Pages to deploy from main and the root folder. 5. Enable static publishing with .nojekyll. 6. Check the published pages and asset responses.

Deployment verified. The public website and its eight HTML pages were checked after deployment. Gallery files and shared styles were also reachable. The site uses HTTPS.

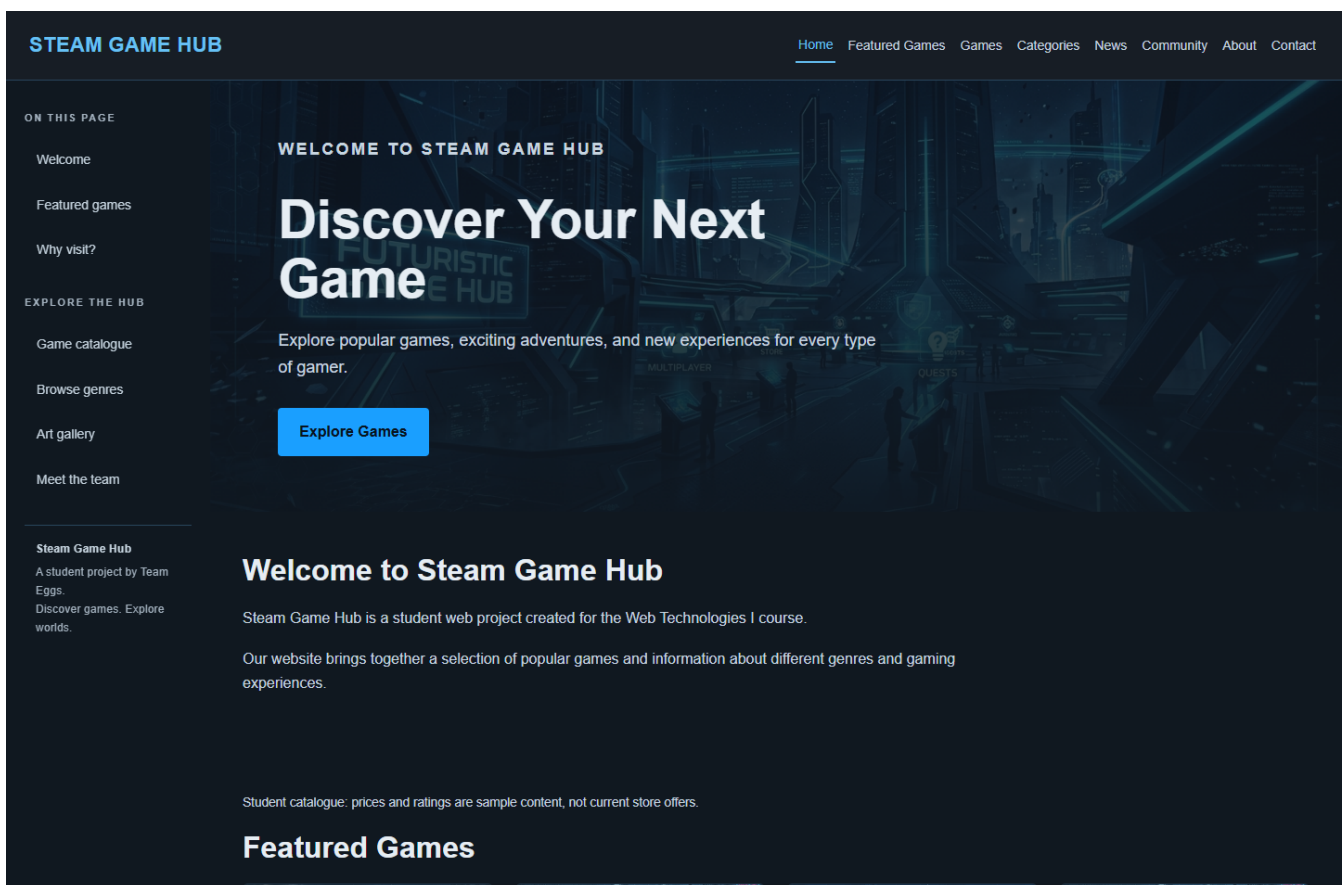


Figure 6. The Home page loaded from the public GitHub Pages address.

The site uses relative links and local assets, so it also runs by opening index.html directly. No package installation, API key, framework or build command is required.

Submission includes this PDF and a ZIP of the full project. Each team member must submit the files and deployed URL in the LMS by the course deadline and defend the work during the practical lesson.

What the project demonstrates

Layout choice. Grid gives the website a predictable two-dimensional structure. Flexbox provides straightforward alignment inside that structure: logo and menu, rows of cards, form fields and smaller components. Combining them avoids using fixed positioning or floats for the page layout.

Responsive design. Flexible widths, minmax, wrapping and media queries allow the same content to adapt to different screens. The sidebar moves above the main area, card rows reduce their column count, and a wide comparison table scrolls inside its own container.

Accessibility. Every page provides a skip link, semantic regions, a current-page indicator and visible keyboard focus. Gallery captions can be revealed with the keyboard, are always available on touch devices, and images include alternative text. Reduced-motion preferences disable decorative movement.

Project boundaries. This is a static educational website. Catalogue prices and ratings, news stories and community counts are demonstration content. Forms validate locally and do not store or send personal information. Existing Assignment 1 artwork and team portraits were retained; nine original SVG illustrations were added for the gallery.

Key points for the practical defense

Concept	What to explain or demonstrate
Flexbox axes	Main vs cross axis; justify-content and align-items.
Card alignment	Stretch, flex-direction: column, flex: 1 and margin-top: auto.
Named Grid areas	How repeated names span columns; how each region gets its grid-area.
Responsive rules	Why the 1200px, 1000px and 620px breakpoints change the layout.
Gallery interaction	Equal cells, aspect ratio, overlay positioning, hover and focus.

References

1. Course brief: Web Assignment 2.pdf, Assignment #2 - Advanced CSS: Flexbox & Grid, pages 2-5.
2. Project source: the repository linked on page 1.
3. [GitHub Docs - REST API endpoints for GitHub Pages](#), used to configure and inspect deployment.